

Find the Missing and Repeating Number

Company: Deloitte

Round: Offline

Year: 2022

Difficulty: Medium

Topic: Arrays, Hashing, Mathematics, Optimization

Source: <https://www.designa.in/9347/hashtedin-deloitte-interview-experiences-internship-campus>

Problem Description

Given an array containing numbers from 1 to N, **one number is missing** and **one number appears twice**.

Find:

- The **missing number**
- The **repeating number**

Example

Input:

[1, 2, 2, 4, 5]

Here, 3 is missing and 2 is repeated.

Output:

Missing = 3

Repeating = 2

Input Format

- The first line contains an integer N, representing the size of the array.
- The second line contains N integers.
- The array contains numbers from 1 to N, where one number is missing and one number is repeated twice.

Output Format

Print two values:

Missing Repeating

where:

- The first value is the **missing number**.
- The second value is the **repeating number**.

Sample Input

5
1 2 2 4 5

Sample Output

3 2

Explanation

Given the input:

5
1 2 2 4 5

The numbers should normally be from 1 to 5:

1 2 3 4 5

But in the given array:

1 2 2 4 5

2 appears twice, so 2 is the repeating number.

3 does not appear, so 3 is the missing number.

Therefore, the output is:

3 2

Here:

3 → Missing number

2 → Repeating number

Approach to Solve This Problem:

1. Nested Loops

- Check each number from 1 to N.
- Count how many times it occurs in the array.
- Occurs **twice** → repeating number.
- Occurs **zero times** → missing number.

2. Hashing

- Use a frequency array/HashMap to store the count of each number.
- Find the number with frequency 2 and the number with frequency 0.

3. Mathematical Approach

- Calculate the expected sum of 1 to N.
- Calculate the actual sum of the array.
- Use the difference between them along with the **sum of squares** to determine the missing and repeating numbers.

Java Solution:

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         long sum = 0;
7         long squareSum = 0;
8         for (int i = 0; i < n; i++) {
9             long x = sc.nextLong();
10            sum += x;
11            squareSum += x * x;
12        }
13        long expectedSum = (long) n * (n + 1) / 2;
14        long expectedSquareSum = (long) n * (n + 1) * (2L * n + 1) / 6;
15        long difference = sum - expectedSum;
16        long squareDifference = squareSum - expectedSquareSum;
17        long sumOfNumbers = squareDifference / difference;
18        long repeating = (difference + sumOfNumbers) / 2;
19        long missing = sumOfNumbers - repeating;
20        System.out.println(missing + " " + repeating);
21        sc.close();
22    }
23 }
```

Solution:

The **mathematical approach** uses two equations to find the missing and repeating numbers.

Let:

- R = repeating number
- M = missing number

Step 1: Find the difference of sums

Expected sum of numbers from 1 to N:

$$N \times (N + 1) / 2$$

Actual sum = sum of all array elements.

Therefore:

$$R - M = \text{Actual Sum} - \text{Expected Sum}$$

Step 2: Find the difference of squares

Expected sum of squares:

$$N \times (N + 1) \times (2N + 1) / 6$$

Actual square sum = sum of squares of all array elements.

Therefore:

$$R^2 - M^2 = \text{Actual Square Sum} - \text{Expected Square Sum}$$

Since:

$$R^2 - M^2 = (R - M)(R + M)$$

we can calculate:

$$R + M = (R^2 - M^2) / (R - M)$$

Step 3: Calculate both numbers

Now we have:

[LinkedIn](#) | [YouTube](#) | [Instagram](#)

$R - M$

$R + M$

So:

$R = ((R - M) + (R + M)) / 2$

and:

$M = (R + M) - R$

Complexity Analysis:

Time Complexity: $O(N)$

1. We traverse the array **only once**.
2. During this traversal, we calculate:
 - Actual sum
 - Actual sum of squares
3. The remaining calculations are simple arithmetic operations taking $O(1)$ time.

Space Complexity: $O(1)$

- We only use a few variables such as actualSum, actualSquareSum, difference, sum, missing, and repeating.
- No extra array, HashMap, or other data structure is used.